

Alpaca Options Trading Agents

Miramar Labs · Alpaca AI Trading Agents Hackathon · lablab.ai · Sep 2026 github.com/miramar-labs/alpaca-options-trading-agents

Paper account **PA3MAA2HJLUG** · \$100k

A three-agent trading floor that autonomously trades US-equity **options**. An **Analyst** sets the daily universe; a **Dealer** produces a BUY / HOLD / SELL signal per symbol every 600 seconds and hands each non-HOLD to an **Alpaca MCP tool-calling agent** that browses the live option chain and picks one specific contract; a **Floor Broker** re-validates that pick server-side and executes it. It runs live and continuously on a Kubernetes cluster against a dedicated Alpaca paper account — not a backtest or a one-off demo.

```
Analyst (CronJob 08:55 ET) → portfolio ConfigMap → Dealer (poll loop, 600s) → BUY/HOLD/SELL (LLM) → non-HOLD →
Alpaca MCP loop (chain → contract) → Floor Broker (FastAPI) → Alpaca: re-validate, place order, synthetic
SL/TP/DTE exit
```

AI LOGIC

- **Analyst** — a LangGraph pipeline: screens Alpaca movers plus a fixed large-cap list, pulls news and TAAPI technical indicators, reads back its own recent track record from Postgres, and asks an LLM for the day's watchlist with a budget and rationale per symbol.
- **Dealer signal** — per symbol, technical indicators and OHLCV-derived market structure go to an LLM for a BUY / HOLD / SELL call plus a confidence score.
- **Contract selection** (the centrepiece) — `mcp_options.py` launches `alpaca-mcp-server` over stdio (`langchain-mcp-adapters`) with a **read-only** toolset. Given the underlying, the direction (call = bullish, put = bearish; always long) and the config DTE / delta windows, the LLM drives a bounded **≤6-round** tool-calling loop over the live chain, quotes and Greeks and returns a structured pick: strike, expiration, delta, premium, reasoning.
- **Validate → reconcile → re-validate** — reject a wrong-type or never-seen-in-chain contract; overwrite strike / expiration / delta / premium with what the chain actually showed (only the reasoning text is trusted); re-check server-side in Floor Broker against live windows and a fresh ask quote before the order goes in.
- **Local inference only** — Ollama serving `qwen2.5:32b-instruct-q4_K_M` for every decision, on a DGX that runs on 100% solar power; no external LLM API. If the structured-output step ever returns empty, a deterministic `_fallback_pick` selects from the same fetched chain rows under the same gates — a miss degrades to “code skims the chain like a human would,” never to no trade.

RISK GATES

Eight layers sit between “the LLM said BUY” and an order reaching Alpaca — each one deterministic, none of them an LLM call:

- confidence threshold on the signal
- macro-event blackout calendar (FOMC / CPI / jobs / PCE)
- same-symbol stop-loss cooldown
- portfolio win-rate throttle
- daily profit-target / loss-limit halt
- operator kill switch — flipped in config, no redeploy
- DTE / delta window re-validation
- hard notional cap, enforced against the live market rather than the model's claimed price

Once a position is open, Alpaca has no server-side bracket order for options, so Floor Broker runs its own 30-second poll loop that force-closes on a **50% loss**, a **175% gain**, or expiration dropping to **3 days** out — whichever comes first, and regardless of whether opening new positions is currently gated.

ALPACA INFRASTRUCTURE IMPLEMENTATION

- `alpaca-py` for trading, market data, news and the options chain / quotes / Greeks — all against the dedicated paper account (`PA3MAA2HJLUG`).
- `alpaca-mcp-server`, the official MCP server, run locally as a stdio subprocess per Dealer poll with a read-only toolset — the agent can inspect the chain but can never place an order.
- **Floor Broker** is the only component with write access to the Trading API: a stateless FastAPI service that submits the option order over plain REST, then runs three background poll loops — fill detection, synthetic SL/TP/DTE exit, and restart recovery (rebuilding all in-memory tracking from Alpaca and Postgres if the pod restarts mid-position).
- **One NVIDIA DGX Spark** — running on 100% solar power — hosts everything: k3s (four workloads in namespace `alpaca-options-trader`), Ollama, a shared Postgres instance (cross-cycle agent memory, audit trail, DB-backed reconciliation), and a self-hosted GitHub Actions runner doing the whole CI/CD path (push → test + lint → build four images → rolling deploy).
- `config.yaml` is fetched fresh from GitHub every 60 seconds by every pod, so risk thresholds, blackout dates, even the LLM model are a plain `git push` — no rebuild, no redeploy.

RESULTS — 3 SEP 2026 CLOSE (DAY 3)

Activity. Three sessions of 600s Dealer polls — mostly HOLD, ~two dozen BUY signals, a handful of SELL. Those produced **7 option positions**, every one filled within half a minute of the signal. Everything else was stopped by the risk layer — sub-0.6 confidence, duplicates of an open position or an order in flight, and contracts with no chain strike passing the DTE / delta / OI / volume gates.

Open book. 3 long calls, 2 long puts, ~\$8.3k current value, entry deltas 0.43–0.53, 13–43 DTE. Account equity **\$103,709** (+3.7% vs the \$100k open), cash \$95,399; the day-3 session gained **+\$2,532 (+2.5%)**. All 5 open positions are green — best FRVO Sep 20P +48%. Day 1 closed at \$98,641 (−1.4%); the book has since recovered that and added 3.7% over the open.

Exits. The Floor Broker's synthetic exit loop closed its first two positions this session, both on the take-profit rule — Alpaca has no server-side bracket orders for options, so this loop is ours. MSFT 505C and META 585C, **+\$2,270 realised** combined, both winners. Two exits is too small a sample to quote a win rate.

Determinism holding. The layer under the model is visibly doing its job — the confidence, duplicate and contract-quality gates each rejected real signals, and the daily profit-target halt tripped once intraday P/L cleared its threshold. Contract selection runs on `qwen2.5:32b-instruct-q4_K_M`, verified end-to-end through the real MCP selection path (`_fallback=False` on NVDA / TSLA / AAPL) and committed as `scripts/check_contract_selection.py`; see `docs/models.md`. Live P/L and model badges are in the README.